OSDI/Verilog-A Device Bypass — an Investigation

Why element bypass is not shipped for OSDI devices

Ngspice + OpenVAF Enhancements project

July 2026

Contents

OSDI/Verilog-A device bypass — an investigation, and why it isn't shipped

ngspice's native compact-model devices (diode, BJT, MOSFET, ...) implement element bypass (a.k.a. latency exploitation): under `.option bypass` a device skips recomputing its currents/charges/derivatives when its controlling voltages have not moved since the last accepted evaluation, reusing the stored linearization instead. The OSDI/Verilog-A device path (`src/osdi/osdiload.c`) has no such bypass — it calls the compiled `eval()` for every instance on every Newton iteration.

This note records a full attempt to add element bypass to the OSDI path, the measurements that came out of it, and the conclusion: it does not pay off, can degrade robustness, and is deliberately not shipped. The prototype was built, verified for correctness, measured, and then reverted. This is the same shape of result as the compile-time deep-dive — a plausible optimization that careful measurement does not support.

TL;DR

	Result
Is OSDI bypass correct?	✓ Yes — when it converges, a bypass-on run matches bypass-off to max rel. error 3.8e-7 on a 40-transistor BSIM4 transient.
Is it faster?	× No — it was ~5 % slower on that same circuit, and ~35 % slower with tightened tolerances.
Why no speedup?	OSDI `eval()` is one monolithic compiled function, so the per-device residual + Jacobian needed for a *safe* bypass test must be extracted with extra `.osdi` calls on every full eval — overhead that native devices don't have (their `g/c` fall out of `load()` for free).
Worse than overhead	Freezing bypassed devices' linearizations degrades Newton convergence: measured 1.6× more total iterations, so the absolute number of full evals *rose* even though 34 % of calls were bypassed.
Robustness	× On a circuit with many quiescent devices, enabling bypass caused an outright transient convergence failure (timestep collapse at t≈0) where bypass-off converges cleanly.
What ngspice itself does	Ships `.option bypass` disabled by default (`CKTbypass = 0`, `src/spicelib/devices/cktinit.c`) — consistent with these findings.
Decision	Not shipped. Leaving `.option bypass` as a harmless no-op for OSDI devices is safer than wiring it into a footgun.

What element bypass is, and the safe way to do it

At a Newton iteration a device is asked to stamp its contribution to the Jacobian and RHS at the current node-voltage guess. Bypass observes that if the device's terminal voltages have barely moved since its last full evaluation, the previously computed contribution is still valid, so the expensive model evaluation can be skipped and the stored values re-stamped.

The naive version — "skip when the voltages moved less than a tolerance" — is unsafe. On a steep operating point (a MOSFET in subthreshold, a forward diode) a *within-tolerance* voltage move still swings the current by a large factor; reusing the stale linearization there makes Newton oscillate and the timestep collapse. ngspice's native devices therefore gate bypass on two tests (see `src/spicelib/devices/dio/dioload.c`):

1. every controlling voltage moved less than `voltTol + reltol·|V|`; and
2. the *predicted current* at the new voltage, `cdhat = cd_old + g_old·ΔV`, is within `abstol + reltol·|cd|` of the stored current.

The second gate is the one that matters — it refuses bypass exactly where the model is nonlinear enough that a frozen Jacobian would break convergence.

The prototype

The reverted prototype (`osdiload.c` / `osdidefs.h` / `osdisetup.c`) implemented the faithful generic form of both gates for OSDI:

- Skip `descr->eval()` when the instance is bypass-eligible; the eval output buffers persist in the instance data, so the existing `load()` re-stamps the still-valid values — exactly how native devices reuse their stored `g`/`c`/`q`.
- Gate 1 (voltage): loop the mapped nodes, compare `CKTrhsOld[node]` against a per-instance snapshot `bypass_volts[]`.
- Gate 2 (current): the generic form of the `cdhat` test. Using the resistive Jacobian captured at the last full eval (`write_jacobian_array_resist`) and the jacobian-entry row/col map, predict each node's stamped-current change `dI[i] = Σ_k J[k]·ΔV[col_k]` and require `|dI[i]| < abstol + reltol·|residual[i]|`, with the per-node residual captured via `load_residual_resist`.
- Phase guard: the persisted buffers are only reusable when the *current* eval would run with the same `OsdiSimInfo.flags`; a mismatch (DC↔tran, the `ANALYSIS_STATIC`/`ANALYSIS_IC` init step, AC, noise) forces re-evaluation. Without this, the DC-operating-point buffers get reused for the first transient point at an unchanged voltage, stamping a static solution where the reactive (charge) contribution is required — a nonphysical result.
- Exclusions: AC / small-signal (fresh linearization), the predictor step (`MODEINITPRED` — this guarantees one full eval per transient timepoint before any bypass), and any model using the time-history builtins `absdelay` / `last_crossing` (whose eval depends on time, not just voltage).

This prototype is correct. On a 40-transistor pseudo-NMOS BSIM4 inverter chain, a bypass-on transient reproduced the bypass-off waveforms to a maximum relative error of 3.8e-7 — i.e. the two gates do their job: bypass fires only where it is genuinely harmless.

Why it doesn't pay off

1. The safety test is not free for OSDI

Native devices compute `g`, `c`, and the branch current inline during `load()` and stash them in the state vector as a byproduct, so their bypass test reads values that already exist — nearly zero cost. An OSDI model's `eval()` is a single monolithic compiled function; the individual resistive residual and Jacobian are not exposed as a free byproduct. To run gate 2 the prototype must call `load_residual_resist` and `write_jacobian_array_resist` on *every full eval*, then run the entry-map accumulation loop on every candidate.

Isolating that cost (force bypass to never fire, so every call pays the store + test but skips nothing) measured ~7 % overhead on the inverter chain — pure bookkeeping, no eval saved.

2. Bypass degrades Newton convergence

More damaging: a bypassed device contributes a frozen Jacobian entry while the rest of the circuit is still moving. That demotes Newton from quadratic toward linear convergence, so the circuit needs more iterations per timestep. On the inverter chain (tightened tolerances, to make the effect measurable):

	total eval-candidate calls	rows	full evals actually run
bypass off	1,445,880	12,060	1,445,880
bypass on (34.4 % bypassed)	2,306,080	12,121	1,512,788

Bypass caused 1.6× more total iterations. Even after skipping 34 % of them, the absolute number of full evals rose (1.51 M vs 1.45 M). The device evaluation is ~58 % of Newton time (profiled), so more evals means more time — the run went from 1.45 s to 1.96 s (+35 %). At default tolerances the effect is milder but still net-negative (1.23 s → 1.30 s, ~5 % slower).

3. It can break convergence outright

On a circuit combining a few hard-switching stages with ~200 DC-biased quiescent BSIM4 cells, enabling bypass made the transient abort at t≈2e-13 with "timestep too small" — while the identical bypass-off deck completes 10,068 timepoints. When a large fraction of devices bypass simultaneously, the Jacobian becomes too stale for Newton to correct, and the step controller grinds the timestep to the floor. Both gates were active; they bound per-device error but not the *aggregate* convergence damage.

Why native devices tolerate what OSDI can't

Native and OSDI devices suffer the same convergence degradation from frozen linearizations. The difference is the cost side of the ledger: for native devices the bypass test is essentially free, so even a modest iteration increase is offset by the free eval-skips, and the feature is a small win on the circuits where it helps. For OSDI the test carries real per-eval overhead (§1), so the same iteration increase (§2) is *not* offset — and on top of that the aggregate staleness can cross into non-convergence (§3).

This is also why ngspice ships `.option bypass` off by default (`CKTbypass = 0` in `cktinit.c`; `TSKbypass = 0` in `cktntask.c`): bypass is a known accuracy-and-robustness trade that helps only some circuits even for native devices. Wiring it into the OSDI path would convert a currently-harmless no-op into an option that can silently slow down or break a simulation when a user enables it.

Conclusion

Element bypass for OSDI/Verilog-A devices is implementable and correct, but measurement shows it delivers no speedup (typically a slowdown) and can degrade robustness to the point of convergence failure, because:

1. the per-device residual/Jacobian extraction required for a *safe* bypass test is not free for a monolithic OSDI `eval()`; and
2. freezing device linearizations inflates the Newton iteration count enough to erase — and often invert — the eval savings.

The prototype was therefore reverted; `.option bypass` remains a no-op for OSDI devices, matching ngspice's own decision to keep bypass disabled by default.

Reproducing

- Prototype: add `bypass_volts` / `bypass_resid` / `bypass_jac` / `bypass_flags` to `OsdiExtraInstData`; in the serial `OSDIload` loop skip `eval()` when a two-gate `osdi_bypass_ok()` (voltage + Jacobian-predicted current, both against the ngspice `abstol`/`reltol`/`voltTol`) passes, capturing the residual/Jacobian via `load_residual_resist` / `write_jacobian_array_resist` after each full eval.
- Correctness: run a device-heavy BSIM4 transient with `.option bypass=0` and `.option bypass=1`, interpolate both onto a common time grid, and compare — the waveforms agree to `reltol`.
- Overhead / iteration counts: instrument the loop with hit/total counters and a force-off switch; compare eval-candidate totals and wall time across bypass-off, bypass-forced-off (store+test but never skip), and bypass-on.
- Robustness: a deck mixing a few fast-switching stages with a large bank of DC-biased quiescent devices exposes the aggregate-staleness convergence failure.